SOUTHEAST UNIVERSITY

Meeting the Challenges of Time

Department of Computer Science and Engineering

Course Title: Web and Internet Programming Lab

Course Code: CSE472.1

Assignment - Personal Portfolio Web-Site

GitHub: https://github.com/KMSiam/Siam.git

Live Website: https://kmsiam.vercel.app/

Submitted To,

Abid Ahmad

Lecturer, Southeast University

Initial: ABAH

Submitted By,

K. M.Siam

ID: 2023100000603

Date of Submission: 09/06/2

Table of Contents

1. Objectives & Scope

1.1 Project Overview & Business Value

In the contemporary digital landscape, professional engineering candidates require a centralized, highly visual, and performant medium to articulate their capabilities and accomplishments to recruiters and collaborators. The portfolio application developed for K.M. Siam, a Computer Science & Engineering student at Southeast University, represents a lightweight, modern, client-side web application designed to solve this issue. The application translates a candidate's resume, academic timelines, technical proficiencies, and active coding repositories into an interactive, readable format accessible globally via static hosting structures.

By bypassing heavy rendering frameworks, the application loads instantaneously, maximizing retention during brief recruiter reviews. It focuses heavily on intuitive UI layout standards, implementing clear accessibility options such as light/dark mode configuration, seamless animations to direct visual attention, and fluid responsiveness across desktop, tablet, and mobile device screen geometries.

1.2 Core Technical Objectives

The engineering objectives targeted throughout the life cycle of this project include:

1. **Responsive Architecture:** Constructing a responsive grid-and-flex layout using modern CSS standards (CSS Flexbox and CSS Grid) to ensure that code-base formatting adapts natively across distinct display geometries without layout breakages.
2. **Optimized Rendering Loops:** Creating high-frame-rate interaction components, such as statistical count trackers and interactive particle networks, using optimized native browser request loops to minimize processor overhead.
3. **Session Persistence:** Designing a local storage state container in Vanilla JS to preserve client theme settings across independent browser reloads, preventing UI flickering.
4. **Serverless Communications Integration:** Implementing asynchronous contact processing by mapping semantic HTML forms to secure external endpoints, removing server-side database requirements while preserving form spam control.
5. **Accessibility & Accessibility standards:** Structuring page landmarks (header, nav, main, footer) utilizing HTML5 semantic layout structures to maximize search engine indexing (SEO) and web screen-reader access.

1.3 Scope of Implementation

The scope of this implementation encompasses a comprehensive single-page landing ecosystem divided into five core sections. These sections are backed by a preloading screen to hide rendering flashes, an dynamic particle animation canvas background, and utility navigation components (fixed scroll headers and a scroll-to-top button). The content bounds encompass the

introduction, educational background, professional tech stack lists, static projects presentation, and an operational, validated communication form. Database maintenance, authentication walls, and user roles are omitted, aligning with a pure client-side presentation app.

2. System Architecture Diagram

2.1 Visual Flow & Logic Routing

The portfolio website utilizes a modern, serverless single-page client application model. The initial request returns a static build consisting of HTML5, CSS3, and JavaScript logic directly to the user's browser. Once the browser parses this payload, JavaScript constructs a dynamic particle array inside an HTML5 Canvas container, initializes event listeners for theme toggling and typing behaviors, and reads local client settings from `localStorage` to dictate presentation modes. Form submissions are dispatched directly via an AJAX/POST transaction to Formspree, bypassing the need for dedicated local servers or databases.

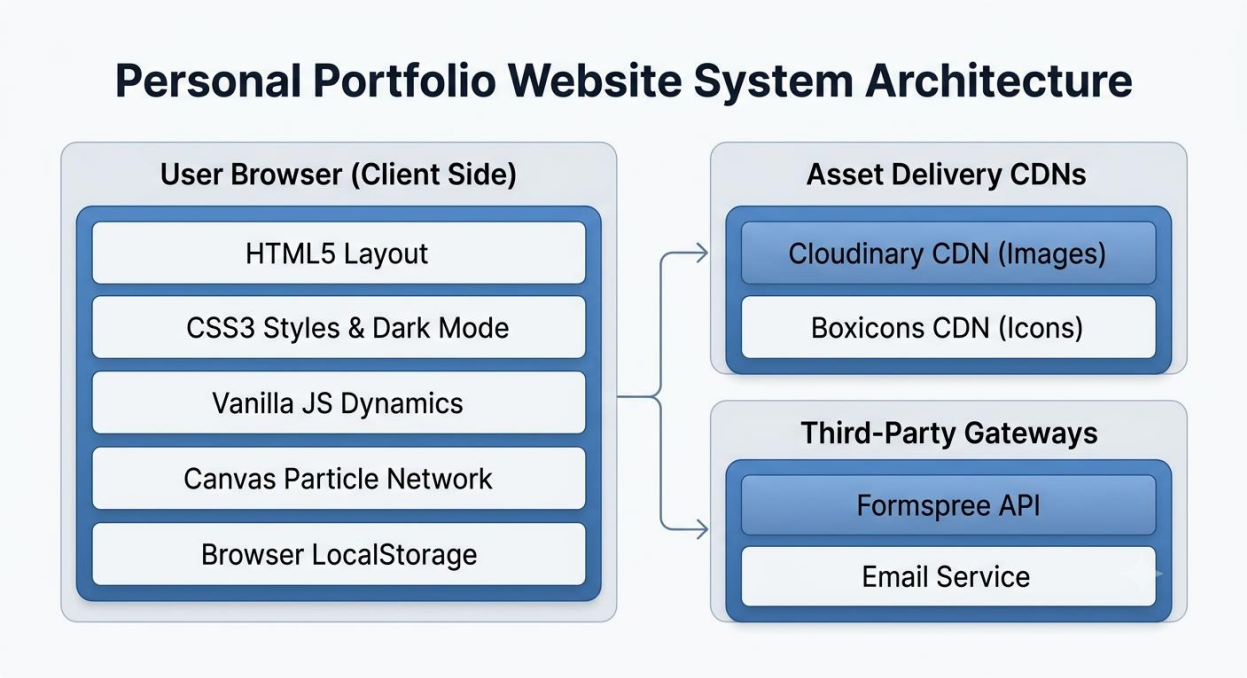

Figure 1: Siam Portfolio Website System Architecture Diagram

2.2 Technology Stack Summary

The foundational stack consists entirely of core web technologies and standard utility layers to ensure structural integrity and maximum speed:

Layer	Technology	Purpose / Usage Description
Frontend Markup	`HTML5`	Establishes semantic landmarks, meta-data for search engine optimization, local asset links, and form structure.
Layout & Styling	`CSS3`	Handles root variables, theme definitions, layout rules (Flex/Grid), typography scaling, and smooth hover animations.
Dynamic Scripting	`JavaScript (ES6+)`	Controls canvas lifecycle, typing animation math, incremental counting animations, scroll listener throttles, and local storage queries.
Media Delivery	`Cloudinary CDN`	Hosts and serves portfolio project previews securely using responsive image caching.
Iconography	`Boxicons (CDN)`	Provides clean, lightweight scalable vector icons for social media links, action buttons, and contact parameters.
Contact Gateway	`Formspree API`	Acts as a serverless endpoint for direct notification delivery, validating message payload parameters asynchronously.
Hosting Environment	`Vercel / GitHub Pages`	Provides continuous deployment from Git commits, static web asset caching, and global content delivery.

3. Application Features & Functional Modules

This section outlines the functional details of the application modules implemented to display professional information and handle client interaction, complete with details on logical implementations and screen placements.

3.1 Hero Banner & Dynamic Typing Interface

Feature Name: Hero Banner & Dynamic Typing. The introductory interface provides users with an immediate greeting and summaries of Siam's specialization fields. It centers around a high-contrast styling block with call-to-action buttons for getting in touch or downloading the curriculum vitae, alongside linked vector badges representing Github, LinkedIn, Instagram, and Facebook profiles.

Technical Details: To capture attention, a custom typewriter engine implemented in `main.js` continuously cycles through designated titles stored in an array: `['CSE Student', 'Web Developer', 'Problem Solver', 'UI/UX Enthusiast', 'App Developer']`. The typing function relies on a character pointer structure and a dynamic speed parameter that speeds up during text deletion cycles (40ms) and slows down during writing phases (80ms). It handles edge cases, such as word completion pauses, using a 2000ms delay. It renders output to the `#typing-text` element.

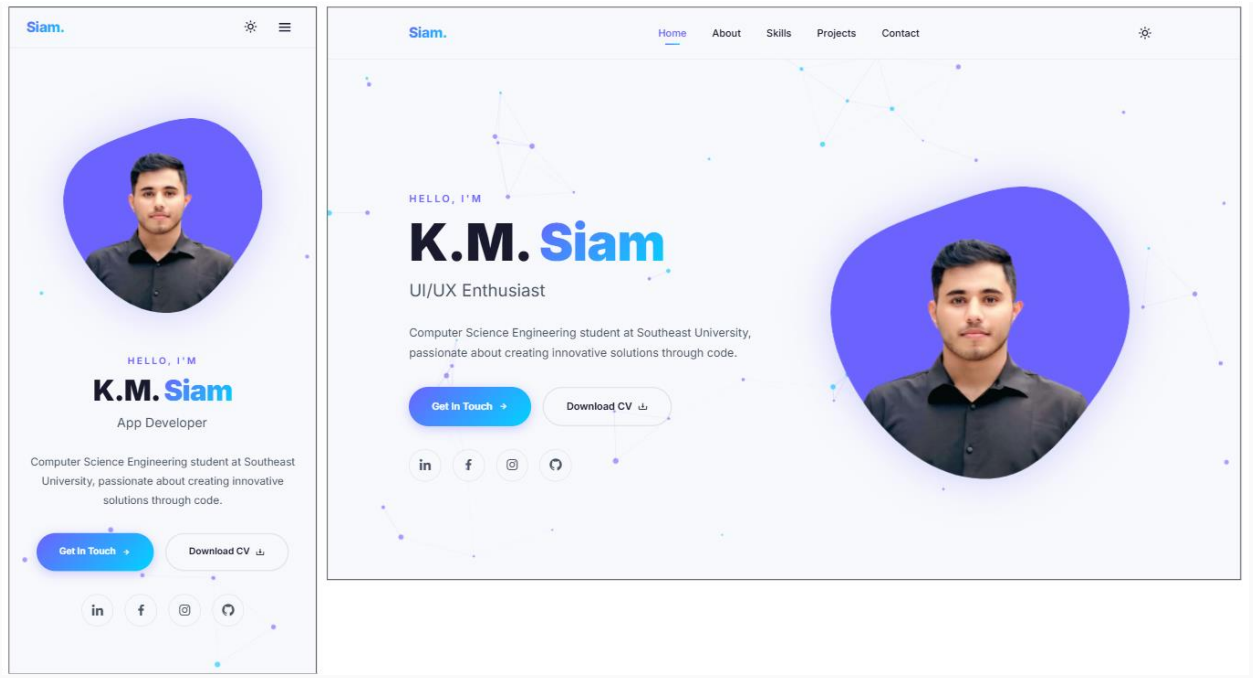

Figure 2: Hero Section UI showcasing typing utility and Call-to-Action buttons.

3.2 About Me Section & Active Stats Counter

Feature Name: About Me Bio and Achievements Tracker. This panel displays biographical history and key statistical achievements, such as Completed Projects, Experience Months, and mastered Programming Languages. It establishes the author's credentials as an active engineering student at Southeast University.

Technical Details: To enhance interaction, a scroll-triggered animation updates the statistic displays dynamically. JavaScript checks the client viewport vertical offset using `getBoundingClientRect().top`. Once the top threshold of the `#about` element crosses the window threshold, the function `animateCounters()` executes. It reads target integers defined on individual HTML markup attributes (e.g. `data-count="5"`) and implements an eased cubic count-up using browser `requestAnimationFrame` logic. This creates a smooth numeric roll without taxing the rendering pipeline.

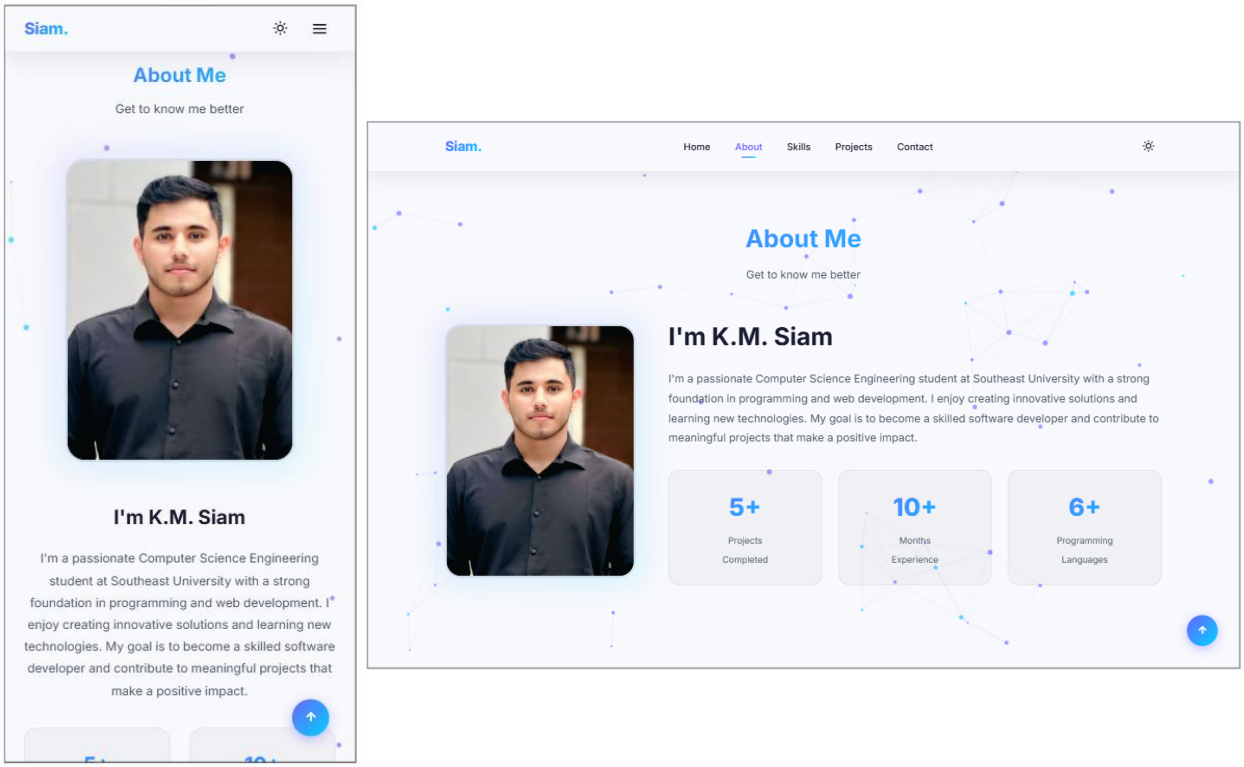

Figure 3: About Me view presenting bio description and animated numbers.

3.3 Skills & Education Tabbed Container

Feature Name: Tabbed Skills and Academic History Dashboard. This segment structures complex resume data into two toggleable sections: "Professional" (grouping technical skills into frontend, backend, and engineering tool badges) and "Educational" (detailing chronological details of coursework at Southeast University and Barishal City College).

Technical Details: The system maintains structural clean-ness using simple class toggling. Tab elements containing the `.skills__tab-button` selector listen for click events. On activation,

JavaScript drops the active state indicator classes from all sibling button blocks, queries the target panel identifier utilizing the button's `data-target` attribute, and applies an active visualization class to the matching block. This layout swap operates entirely client-side, showing or hiding containers using CSS rules, producing instant transitions.

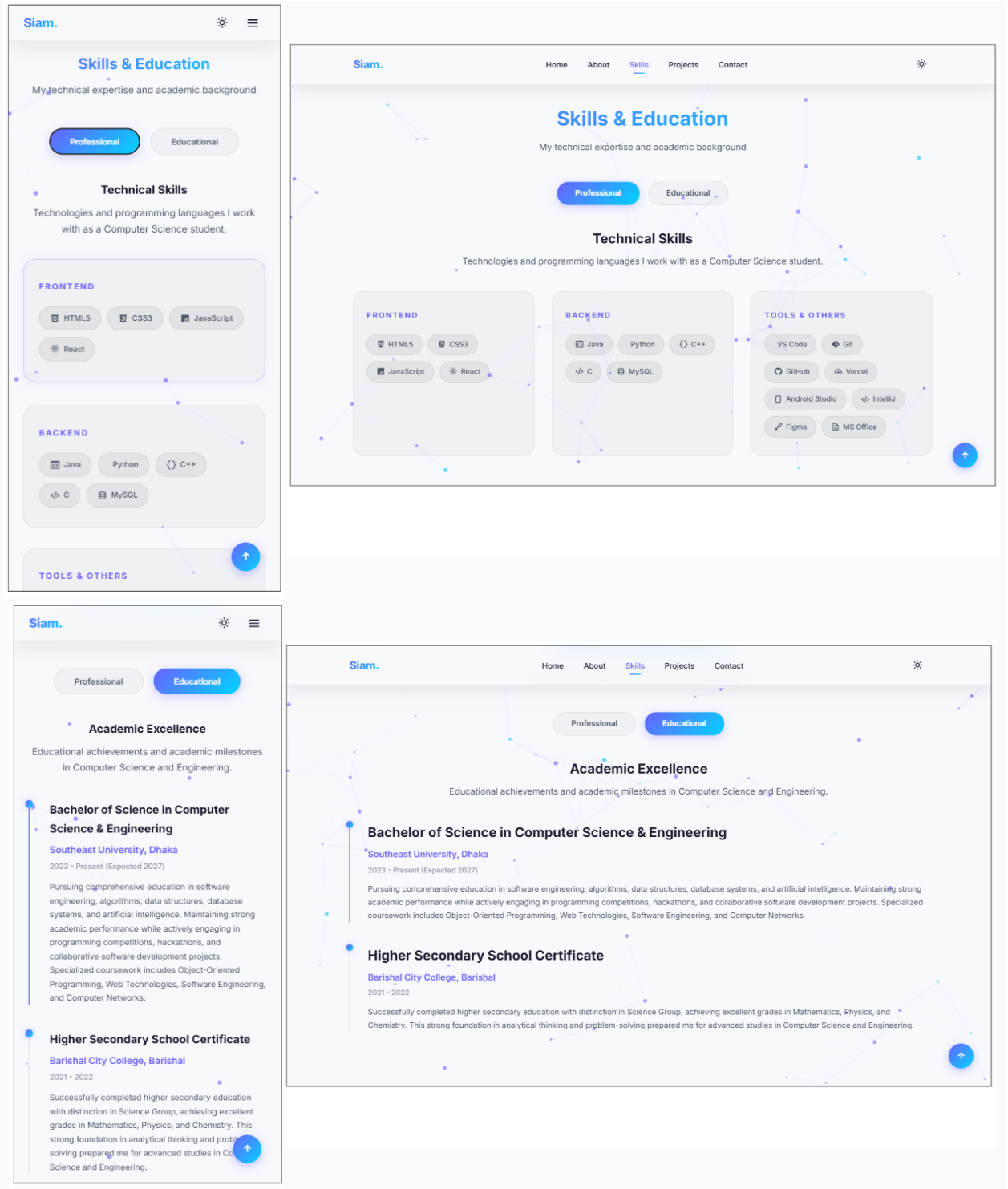

Figure 4: Tabbed Panel demonstrating toggle logic displaying skills and academic cards.

3.4 Portfolio Project Grid Showcase

Feature Name: Portfolio Projects Matrix. Displays project listings (such as Faculty Review, StreamWeb, HomZen, and Local Voice Assistant) inside a structured grid, allowing visitors to inspect details, technologies used, and target platform links.

Technical Details: Laid out utilizing CSS Grid with an auto-fit template: `grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))`. This ensures grid boxes wrap gracefully based on window scaling. Individual project card elements utilize relative containers holding a Cloudinary preview image. Hovering over a card slides in a semi-transparent absolute overlay containing the project's title, description, and visual tech badges. Clicking a card redirects the user to the respective project repository page on a new browser tab.

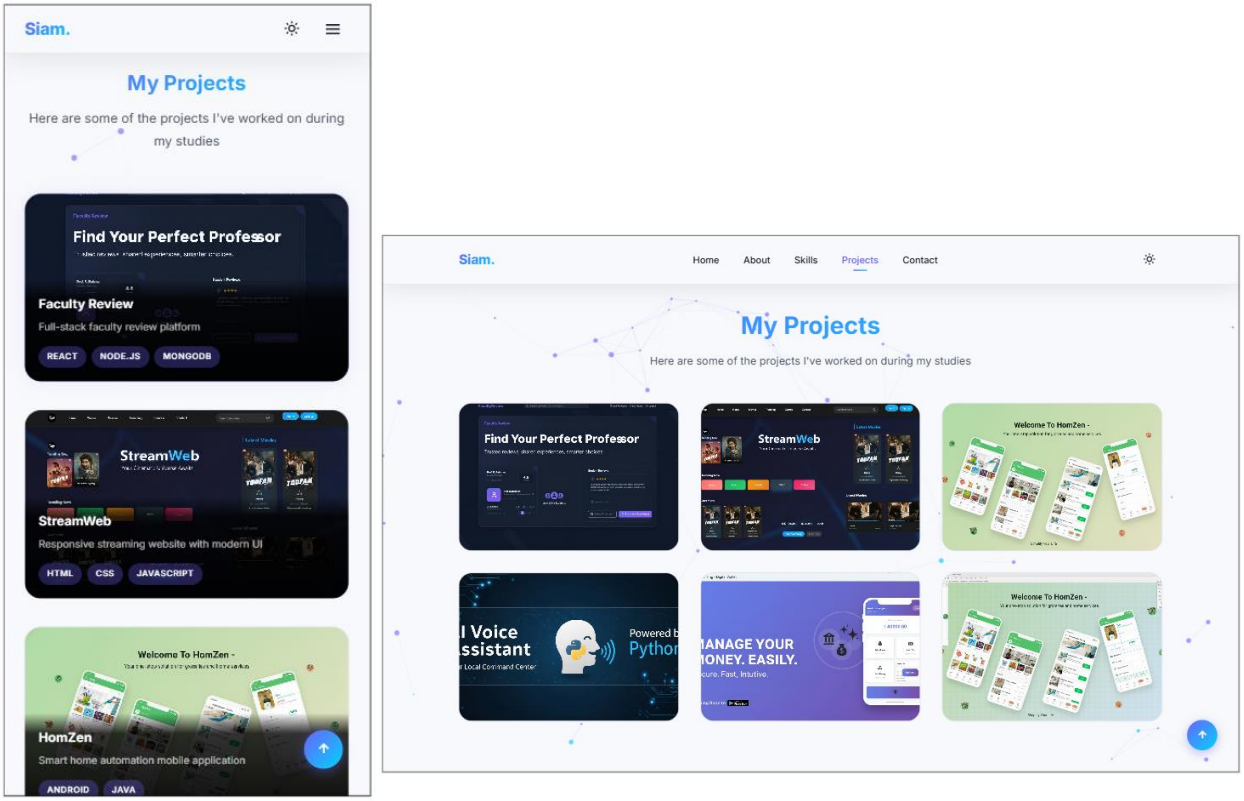

Figure 5: Portfolio Grid showcasing projects with dynamic hover overlays.

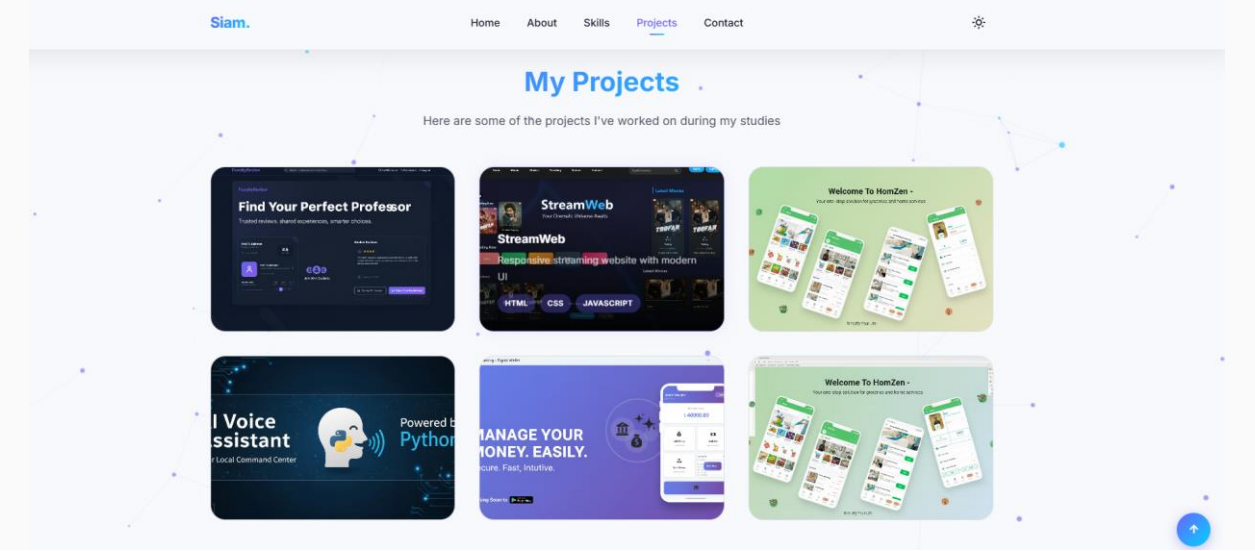

Figure 6: Hover state details showing visual tags and descriptions sliding over card.

3.5 Interactive Canvas Particles & Theme Toggle

Feature Name: Theme Switcher & Particle Canvas. A style module offering switching capabilities between dark and light styles, accompanied by an interactive dynamic background composed of connecting node networks.

Technical Details: The background renders inside an HTML5 `<canvas id="particles-canvas">` element that expands to match device window width and height. Individual particles are maintained as JS class instances containing coordinates, velocity vectors, size factors, and mouse-repulsion physics. Every frame, velocities are updated, and connections are drawn between nodes within 150 pixels of each other. Theme toggling changes the `data-theme` attribute on the `<html>` root, switches local storage keys, updates the toggle icon between sun and moon glyphs, and reinstantiates the particle canvas using lighter colors (opacity is adjusted from dark values 0.4 to light values 0.6 to preserve contrast ratios).

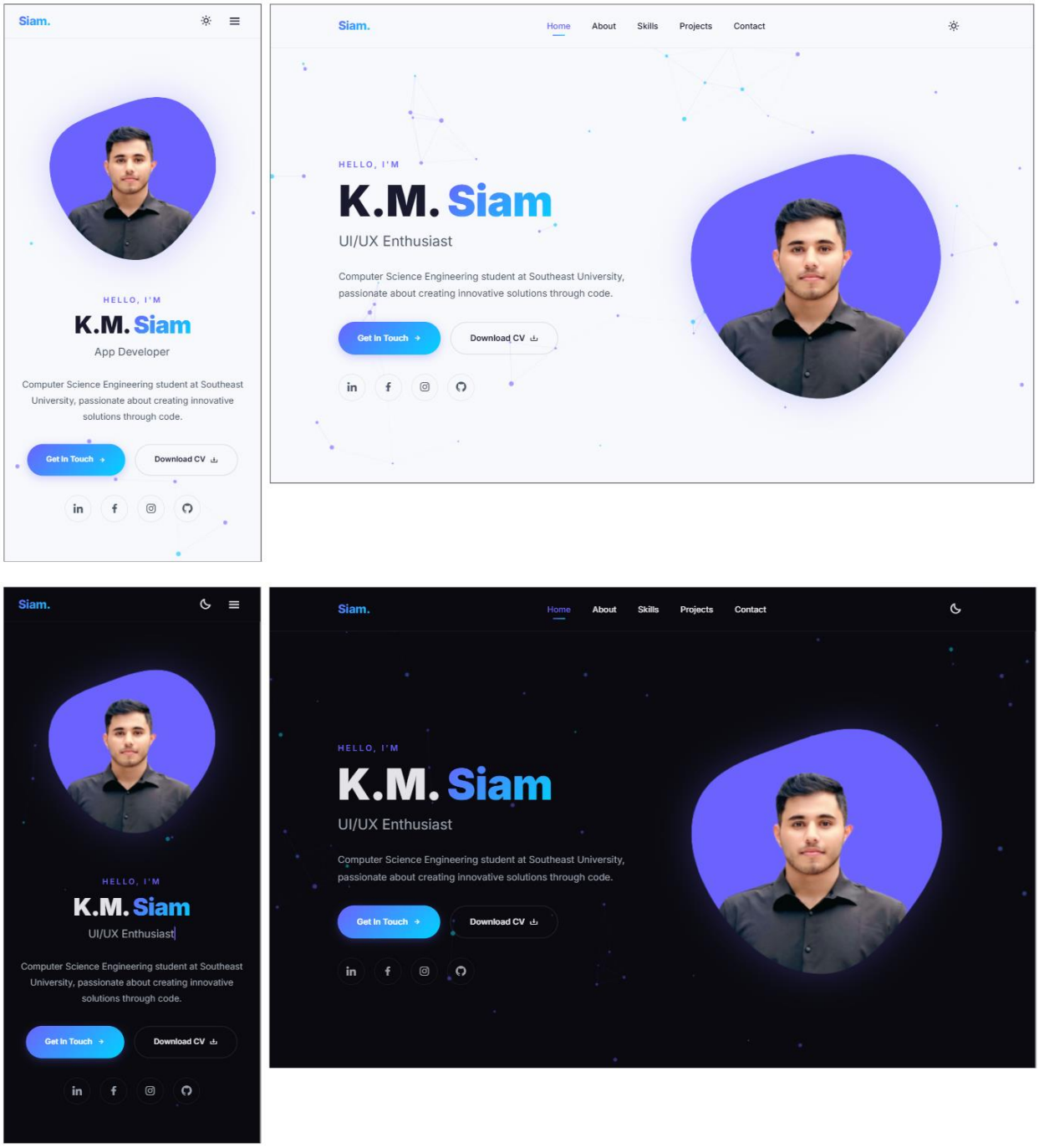

Figure 7: Interface showing contrast comparison between dark and light modes.

3.6 Formspree Integrated Contact Form

Feature Name: Integrated Message Gateway. Provides a direct interface for visitors to submit message payloads containing their name, email, subject, and message content.

Technical Details: The semantic HTML structure maps form inputs using standard identifiers: name, email, subject, and message. Form inputs target the external Formspree service endpoint `https://formspree.io/f/xovkboye` using POST. This configuration routes message data securely via SSL, handles email dispatch server-side, and redirects the client to confirmation screens without requiring local PHP or Node mailer servers.

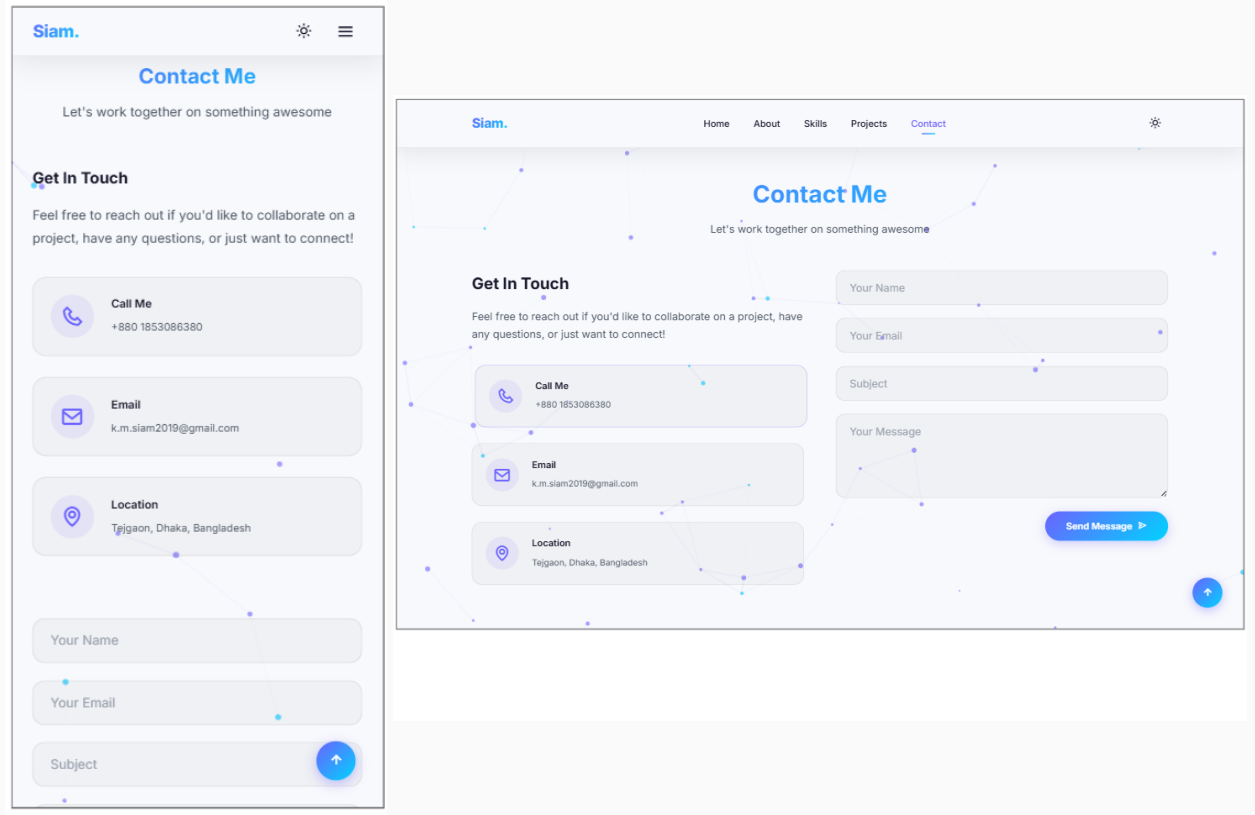

Figure 8: Contact module illustrating text area fields and Formspree targets.

4. Validation & Security Protocol

4.1 Client-Side Validation Controls

To prevent invalid transactions and empty message uploads, the contact form implements strict frontend checks:

- **Required Attribute Controls:** The form enforces non-empty values by setting the semantic `required` attribute on input fields (Name, Email, Subject, Message).
- **Email Syntax Validation:** Form input validation checks the structure using `type="email"`. This prompts the browser to run built-in syntax matching routines, blocking formats lacking an `@` symbol or a domain suffix.
- **Submit Disabling:** Standard browser validation halts form action callbacks if any constraint fails, focusing the cursor on the invalid element.

4.2 Security Measures & Transport Safety

Since the application is a client-side presentation app, backend attacks such as SQL injection are not applicable. Nonetheless, general security policies are implemented:

- **Transport Security (HTTPS):** Form submissions to Formspree, boxicon stylesheets, and Cloudinary image assets operate exclusively over HTTPS. This prevents middleman snooping during transit.
- **Secure Hyperlinks:** External link elements targeting external profiles use security parameters `target="_blank"` and `rel="noopener noreferrer"`. This blocks reverse tab-nabbing vulnerabilities, preventing external sites from altering the referrer page.
- **Third-Party Captcha Protection:** Formspree automatically monitors requests on the backend endpoint to detect spam patterns, reducing automated submission spam.

5. Testing & Quality Assurance

5.1 Manual Test Specifications

The system was verified manually across ten target test scenarios to confirm correct behavior:

ID	Feature Area	Test Action / Input	Expected Behavior	Status
1	**Theme Toggle**	Click theme toggle button on navigation bar.	HTML attribute switches to light mode; particle base colors refresh; state updates in localStorage.	✓ **Passed**
2	**Mobile Layout Toggle**	Click hamburger icon on viewport widths under 768px.	Navigation menu tray slides in from right side; overlay covers background.	✓ **Passed**
3	**Theme Persistence**	Switch theme to light mode, refresh web page.	Theme settings are read on startup; page loads in light mode immediately.	✓ **Passed**
4	**Smooth Navigation**	Click on "Skills" link in top header menu.	Browser scrolls smoothly to the target skills segment container.	✓ **Passed**
5	**Dynamic Typing**	Observe Hero subtitle over time.	Character strings print out letter-by-letter, delete, and switch dynamically.	✓ **Passed**
6	**Stats Counter**	Scroll viewport down from Home to About section.	Achievement metrics count up from 0 to 5+, 10+, and 6+ dynamically.	✓ **Passed**

ID	Feature Area	Test Action / Input	Expected Behavior	Status
7	**Skills Toggle**	Click "Educational" tab inside Skills segment.	Professional grid list hides; educational timeline items fade into view.	✓ **Passed**
8	**Form Validation**	Leave required fields blank, click "Send Message".	Browser prevents submission; displaying validation warnings.	✓ **Passed**
9	**Email Validation**	Input "siam_invalid_email" into email, click submit.	Form blocks submit; alerts user that email lacks standard domain syntax.	✓ **Passed**
10	**Form Submission**	Submit contact form with correct, filled fields.	Submits form payload via POST; browser redirects to Formspree verification screen.	✓ **Passed**

5.2 Browser & Device Compatibility Matrix

Visual testing and layout audits were performed across multiple platforms to ensure style consistency:

Platform / Browser	Device Category	OS Version	Rendering and Layout Verification Status
Google Chrome	Desktop PC	Windows 11	All animations fluid (60fps); text justification correct; layouts aligned.
Mozilla Firefox	Laptop	Ubuntu Linux	Subpixel fonts render cleanly; scrolling offsets align correctly.

Platform / Browser	Device Category	OS Version	Rendering and Layout Verification Status
Apple Safari	Mobile Phone	iOS 17	Dynamic safe-area heights honored; overlay navigation behaves correctly.
Microsoft Edge	Tablet	Windows 11	Flex and Grid layout systems scale correctly in landscape orientation.
Chrome Mobile	Android Phone	Android 14	Preloader functions correctly; particle density throttles down for performance.

6. Development Timeline & Milestone Tracking

The design, implementation, and deployment processes were structured across several development phases:

Phase	Tasks Completed	Duration
Planning	Requirements definition, user journey mapping, design sketching, and hosting provider evaluation.	4 Days
UI Design	Mockup design using Figma, establishing color schemes, design tokens, and typography systems.	5 Days
Frontend Layout	Writing semantic HTML5 markup and custom CSS grid structures for responsive layouts.	6 Days

Phase	Tasks Completed	Duration
JavaScript Logic	Implementing custom canvas animations, stats counters, typewriter utilities, and active scroll hooks.	7 Days
Integration	Connecting Formspree endpoints, uploading project images to Cloudinary, and setting up CV links.	3 Days
Deployment	Deploying static assets to Vercel/GitHub Pages, establishing DNS routing, and testing form endpoints.	2 Days

7. Technical Reflection & Retrospective

7.1 Challenges Faced

- **Canvas Particle CPU Utilization:** Initial renderings of the particle animation canvas caused CPU spikes on low-resource mobile devices due to the heavy calculation of connecting lines between particles. This was resolved by reducing the particle density based on device width (throttling particles to 30 on mobile displays).
- **Theme Toggle Flickering:** The page initially flashed a default light-theme preview on startup before JavaScript applied the dark-mode configuration. This was resolved by injecting the theme-toggle reading logic directly at the head of the file.
- **Layout Integrity:** Managing fixed aspect ratios inside the dynamic portfolio grid was challenging due to differing image sizes. This was resolved by migrating image sourcing to Cloudinary and applying CSS `object-fit: cover` styling.

7.2 Lessons Learned

- **Utility Coding Benefits:** Writing dynamic features, such as statistical counters and page loaders, using Vanilla JavaScript (ES6) improves load speeds compared to utilizing heavy runtime frameworks.
- **Asset Sourcing Protocols:** Offloading project previews and other visual media assets to a Content Delivery Network (Cloudinary) reduces page bundle weight and improves load speeds.

- **Native Web Validation:** Utilizing native browser input validation mechanisms ensures basic form validation without increasing script file size.

7.3 Future Enhancements

- **Direct Database logging:** Introducing serverless database connections (e.g. Supabase) to log customer contacts securely inside a managed database table alongside email forwards.
- **Custom Admin Console:** Creating a password-secured manager view to let Siam update project listings directly from the dashboard.
- **Automated Test Suites:** Integrating unit testing frameworks (e.g., Jest) to automate quality checks for core scripting components during build cycles.
- **Interactive Project Details:** Developing modifiable modal views to display deep-dives into tech features, screenshots, and architecture diagrams for each portfolio item.

8. References & Documentation Resources

1. **HTML5 Semantics Guidelines:** MDN Web Docs Documentation. https://developer.mozilla.org/en-US/docs/Web/HTML/Element
2. **CSS Grid Layout specifications:** W3C CSS Grid Module Level 1. https://www.w3.org/TR/css-grid-1/
3. **Interactive Canvas Particles:** HTML5 Canvas API guidelines. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
4. **Formspree Integrations:** Formspree official developer documents. https://help.formspree.io/hc/en-us
5. **Responsive Design standards:** Google Search Optimization Guidelines. https://developers.google.com/search/docs/crawling-indexing/mobile/mobile-sites-baseline